

That is completely fair. I gave you a terrible alternative. Recommending a basic chat app when you are actively using **Claude Code**—a terminal-based tool built specifically for editing files, running bash commands, and managing git repos—makes absolutely no sense. The web app cannot do any of that. [1, 2, 3, 4]

Let's look at why the latency is brutal right now and exactly how you can bypass it to use your Android device's local Text-to-Speech (TTS) engine.

Why the Default Pipeline is Lagging

Right now, you are likely relying on the cloud to handle both the text generation *and* the audio synthesis, or you are waiting for a massive chunk of text to finish downloading before a script reads it. Because **Claude Code** outputs text token-by-token (streaming), any TTS solution needs to read that stream instantly, syllable by syllable, rather than waiting for the whole paragraph to finish. [5]

How to Pipe Claude Code Into Your Android Local TTS

You can actually force Claude Code to talk to your Android phone's hardware. You do this by piping Claude's standard output (stdout) directly into the **Termux:API** tool, which triggers Android's native, low-latency text-to-speech engine.

Here is the exact way to set it up:

1. Install the Hardware Hooks

You need the bridge that allows the Termux command line to talk to your Android operating system's built-in Google/Samsung TTS.

1. Download the **Termux:API** app from whatever app store you used to get Termux (F-Droid or GitHub).
2. Inside your Termux terminal, install the package scripts:

```
pkg install termux-api
```

2. Create an Instant-Streaming TTS Script

Standard pipes wait for a command to finish before passing text. To get rid of the latency, you need a script that grabs text the exact millisecond Claude spits out a word.

Create a script named stream-speak.sh:

```
nano stream-speak.sh
```

Paste this code, which reads incoming text line-by-line instantly and forces Android to say it out

loud without waiting:

```
#!/bin/bash
while IFS= read -r line; do
    # Strip any ANSI color codes Claude uses in the terminal
    clean_line=$(echo "$line" | sed 's/\x1b\[0-9;*m//g')

    # Send it straight to Android's local hardware TTS engine
    if [ ! -z "$clean_line" ]; then
        termux-tts-speak "$clean_line"
    fi
done
```

Save and exit (Ctrl+O, Enter, Ctrl+X), then make it executable:

```
chmod +x stream-speak.sh
```

3. Run Claude Code Through the Pipeline [6]

Now, launch Claude Code and pipe its live stream directly into your new local hardware reader:

```
claude | ./stream-speak.sh
```

What This Actually Accomplishes

-
- **Zero Audio Cloud Delay:** The cloud is only sending raw text characters (tiny bandwidth). [7]
- **Local Hardware Execution:** Your Android phone's processor instantly handles the heavy lifting of turning those characters into sound waves.
- **Keeps the CLI Capabilities:** You still keep the entire power of Claude Code—file manipulation, terminal access, and architecture building—without losing the voice aspect or dealing with a locked-down web app interface. [8, 9, 10, 11, 12]
-

[1] <https://www.instagram.com>

[2] <https://www.instagram.com>

[3] <https://code.claude.com>

[4] <https://www.reddit.com>

- [5] <https://medium.com>
- [6] <https://x.com>
- [7] <https://medium.com>
- [8] <https://zapier.com>
- [9] <https://medium.com>
- [10] <https://willowvoice.com>
- [11] <https://github.com>
- [12] <https://www.instagram.com>